

The FEN Schema Security Architecture

M. A. Thomas

1 Core Principle

The FEN schema operates at the semantic level. Its objects—Facts, Episodes, Narratives—are abstracted from any particular storage technology. The security model must follow suit. The primitive is not “encrypt this PDF” or “encrypt this database row.” The primitive is “protect this hemoglobin Fact,” “protect this oncology Episode,” “protect this patient-authored Narrative.”

This matters because FEN objects do not map one-to-one onto storage objects. A single Episode may draw from many stored records. A single Fact may have provenance in one location, payload data in another, and policy metadata in a third. Tying security to the storage model produces a brittle system whose protection boundaries diverge from its semantic boundaries.

Three levels of access control.

1. **Storage access.** The ability to fetch encrypted blobs and ciphertext from the infrastructure. Unavoidable for any process that touches the storage layer.
2. **Cryptographic access.** The ability to obtain decryption keys. Governed by policy; scoped to specific principals and purposes.
3. **Semantic access.** The ability to reconstruct a plaintext Fact, Episode, or Narrative as a meaningful object. Requires both cryptographic access and satisfaction of the governing consent policy.

The security layer governs semantic access directly and shapes cryptographic access. Its relation to storage access depends on infrastructure choices.

Derivations. Many useful views are assembled on demand from underlying Facts and Episodes. Once we allow recomposition, encryption policy must govern derived objects too. A generated Narrative, timeline, or export inherits constraints from its constituents. The composition rules are specified in Section 7.

2 Threat Model

The architecture must address the following adversary scenarios. They are ordered roughly by likelihood in a production health-data platform.

1. **Curious employee.** A Phoros engineer or operator with access to operational infrastructure queries `FactOperational` records. They should not be able to decrypt payloads without triggering an audit event. This is the most common real-world breach vector in health technology.
2. **Compromised service account.** An automated process with legitimate access to some encrypted payloads has its credentials stolen or misused. Blast radius depends on key scoping—per-person, per-Fact-type, or platform-wide.
3. **Malicious data borrower.** A party with authorized access to a cohort’s data attempts to re-identify individuals, retain data beyond the authorized period, or combine disclosed data with external datasets. This is a contractual and cryptographic problem simultaneously.
4. **Infrastructure compromise.** An attacker gains access to the storage layer—databases, object stores, backups. They see ciphertext and `FactOperational` metadata. Section 6.1 addresses what this reveals.
5. **Colluding insiders.** Two or more Phoros employees, or one employee and one external party, combine their access to exceed what either should have alone. Multi-party key schemes mitigate this.
6. **Legal compulsion.** A subpoena or court order demands patient data. The architecture should make it possible to comply narrowly—producing specific Facts for a specific person—without exposing the broader dataset. Granular encryption supports this.
7. **Phoros itself as adversary.** The patient’s data should be protected even if Phoros as an organization acts against the patient’s interests. This is aspirational and probably not fully achievable in v1, but stating it clarifies where the architecture falls short and where future hardening should focus.

Key management, including key derivation hierarchy, rotation, and compromise recovery, is deferred to a separate document. The present document assumes that keys are policy-governed and scoped, without specifying the mechanism.

3 Eager Materialization

There are two senses in which FEN objects get materialized. *Semantic materialization* means constructing FEN objects from raw inputs (e.g., data ingested from Epic). *Plaintext materialization* means decrypting data so a principal can read it. Since everything is encrypted at rest, plaintext materialization happens every time someone accesses data. The eager-vs-lazy question concerns semantic materialization.

We adopt eager semantic materialization as the default. Incoming source data are translated into canonical FEN objects at ingestion time. The system stores encrypted Facts, Episodes, and other semantic units as the authoritative protected objects, each with a stable identity and associated policy metadata.

The advantage: the object being encrypted and the object being governed are the same object. Access control, auditing, provenance binding, and consent enforcement all attach to entities

that already carry meaning in the system.

The system permits deferred construction only for higher-order derived objects—Episodes or Narratives generated from existing Facts. In these cases, the derived objects inherit protection constraints from their constituents rather than introducing ungoverned semantic material.

4 Facts as the Cryptographic Unit

The central granularity question: what is the smallest unit that can carry independent provenance, consent constraints, and disclosure meaning?

The answer is the Fact. A Fact represents an assertion, measurement, or observation arising from a sufficiently unified act of generation. A lab result, a vital-sign measurement, a diagnosis assertion, a medication administration—each corresponds to a single generating event.

Two criteria determine Fact boundaries:

1. **Unified generation.** The fields of the Fact are jointly produced by a single clinical measurement, observation, or inference process.
2. **Consent separability.** It is sensible to treat the Fact as a single object of disclosure or restriction under a patient’s consent policy. If a component would require independent disclosure rules, it should be a separate Fact.

This produces a natural alignment:

Facts are the cryptographic unit. Episodes are the clinical unit. Narratives are the interpretive unit.

A Fact is encrypted as one payload. Its internal fields stay together because their meaning arises from the unified event that produced them. Encrypting fields separately would fragment the semantic object while multiplying key management complexity.

Episodes and Narratives are protected indirectly through their constituent Facts. An Episode inherits protection constraints from its Facts. A Narrative is governed through the combined constraints of its underlying Facts and Episodes. When the interpretive structure itself carries sensitive meaning beyond the underlying data, the higher-level object may receive additional cryptographic wrapping.

Qualifications. Certain metadata must remain visible outside the encrypted payload to support indexing, routing, and policy evaluation. This is the **FactOperational** layer described in Section 6.

When a candidate Fact contains components with materially different consent implications, the correct response is usually to model them as separate Facts, not to encrypt fields separately within one Fact.

Selective disclosure within a Fact (nested protection, partial payload encryption) should remain exceptional.

5 Plaintext Materialization

Facts do not exist in readable form in the storage layer. The readable object arises when the system reconstructs it through a controlled procedure: evaluate policy, retrieve encrypted fragments, verify integrity, decrypt. Plaintext Facts exist only transiently in memory during authorized access.

Storing or caching plaintext objects in a database would break the alignment between the semantic model and the security model. We do not do this.

The procedure. Every Fact has a persistent `FactId`. The corresponding `FactOperational` record (Section 6) contains what is needed to reconstruct the object: the payload locator, integrity hash, and policy reference. It is a registry entry, not a container for plaintext.

When a principal requests a Fact:

1. The system retrieves the `FactOperational` record for the requested `FactId`.
2. The policy engine evaluates the request against the governing consent conditions, the principal's identity, and the declared purpose of use.
3. If policy evaluation succeeds, the system retrieves the encrypted payload fragments referenced by the `PayloadLocator`.
4. Integrity checks confirm the fragments match the stored `integrity_hash`. [What is being hashed \(encrypted payload bytes, the semantic Fact fields, both\)? What happens to hash after key rotation if payload is re-encrypted?](#)
5. Cryptographic keys are released under the governing policy.
6. Decryption produces the payload, which the system assembles into the typed Fact defined by the FEN schema.

Episodes and Narratives follow the same principle. Their `FactOperational` records (or lightweight operational equivalents) reference multiple constituent Facts. Materialization involves recursive reconstruction of constituents, each subject to its own policy. The resulting object inherits the combined constraints (Section 7).

Two indexes. The architecture separates two forms of indexing. An *association index* maps a `PersonId` to associated `FactId`, `EpisodeId`, and `NarrativeId` values. It supports navigation but contains no clinical payloads. The `FactOperational` records themselves serve as the second index: given a `FactId`, they provide the reconstruction path to the encrypted payload.

6 FactOperational

The platform needs to index, search, discover cohorts, and evaluate policy without decrypting payloads. `FactOperational` is the operational representation that supports this.

It contains: the stable `FactId`, the associated `PersonId`, Fact type, temporal boundaries, coarse classification tags, a reference to the governing policy, a locator for encrypted payload fragments, and integrity verification material. It may also contain derived indexing features or search tokens for cohort discovery.

`FactOperational` is not a subset of the semantic Fact. Some elements (identifiers, temporal boundaries) appear in both. Others (payload locators, indexing metadata) exist only in the operational layer. The full semantic representation contains the decrypted payload that the operational structure intentionally omits.

Different payload types require different operational variants. A `VitalsOperational` variant exposes flags like `has_blood_pressure` and `abnormal_flag`. A `LabOperational` variant exposes `lab_test_code` and a bucketed value range. The operational layer mirrors the `FactPayload` taxonomy while excluding decrypted clinical values.

6.1 What `FactOperational` reveals

`FactOperational` is a selective mitigation of PHI exposure, not a complete elimination. A Phoros engineer debugging an indexing pipeline sees `abnormal_flag: true` on a lab Fact, not “creatinine 4.2 mg/dL.” That is a meaningful reduction in exposure surface for routine operations. It is not zero-knowledge.

The reduction is uneven across Fact types. `VitalsOperational` with `abnormal_flag` is relatively low-risk. `LabOperational` with `lab_test_code = HIV_VIRAL_LOAD` is high-risk even without a value, because the existence of the test is itself sensitive. The existence of an oncology-typed Episode is sensitive. Operational variant design must make sensitivity-aware decisions about which fields to expose

Phoros engineers would have access to all `FactOperational` fields, both low and high risk? Is there a role-based access layer within `FactOperational`, or is the design assumption that all operational metadata is available to any authenticated Phoros process?

Access to `FactOperational` is governed and audited. The fact that payloads are encrypted does not make operational metadata a free-for-all. Access controls on the operational layer are less restrictive than payload decryption, but they exist.

External parties such as data borrowers do not receive access to `FactOperational` fields. Borrowers interact only with aggregate cohort counts until individuals grant explicit permission.

Idea: add sensitivity tier to `FactOperational`

```
pub enum OperationalSensitivityTier {
    Routine,          // abnormal flags, visit counts, generic procedure types
    Elevated,         // specific lab test codes, medication classes
    Restricted,       // HIV, oncology, mental health, substance use codes
}
```

6.2 Layered materialization

All PHI is stored encrypted at rest. The security layer permits two levels of materialization:

1. **Operational materialization.** The `FactOperational` record is accessible to authorized Phoros platform processes. Operational PHI required for indexing, policy evaluation, and cohort discovery is available. The sensitive payload within the `FactPayload` variant remains encrypted.
2. **Full plaintext materialization.** The complete Fact, including payload contents, is reconstructed. Permitted only for the individual account holder or for an authorized recipient acting under permissions granted by that individual.

6.3 Interaction with the query optimizer

The FEN schema’s query optimizer operates over `FactOperational`, not over decrypted Facts. This constrains what queries are expressible without decryption. Subject filtering, time-range filtering, Fact-type slicing, and code-based narrowing all operate on operational fields. Predicate evaluation that requires payload content (e.g., “creatinine above 2.0”) requires plaintext materialization of candidate Facts and can occur only under policy authorization.

6.4 Rust specification

```
use uuid::Uuid;
use chrono::{DateTime, Utc};

pub type FactId = Uuid;
pub type PersonId = Uuid;
pub type PolicyId = Uuid;

#[derive(Debug, Clone)]
pub struct PayloadLocator {
    pub storage_system: String,
    pub object_key: String,
    pub fragment_ids: Vec<String>,
}

#[derive(Debug, Clone)]
pub struct FactOperationalHeader {
    pub fact_id: FactId,
    pub person_id: PersonId,
    pub interval_start: DateTime<Utc>,
    pub interval_end: DateTime<Utc>,
    pub policy_ref: PolicyId,
    pub integrity_hash: String,
    pub payload_locator: PayloadLocator,
    OperationalSensitivityTier // new
}

#[derive(Debug, Clone)]
pub struct VitalsOperational {
```

```

    pub header: FactOperationalHeader,
    pub has_blood_pressure: bool,
    pub has_heart_rate: bool,
    pub abnormal_flag: bool,
    pub classification_tags: Vec<String>,
}

#[derive(Debug, Clone)]
pub struct LabOperational {
    pub header: FactOperationalHeader,
    pub lab_test_code: String,
    pub value_bucket: Option<String>,
    pub abnormal_flag: bool,
}

#[derive(Debug, Clone)]
pub enum FactOperational {
    Vitals(VitalsOperational),
    Lab(LabOperational),
    // Medication, Imaging, Procedure, etc.
}

```

6.5 Operational representations of Episodes and Narratives

The case for operational twins is strongest at the Fact level, because Facts are the primary units of ingestion, indexing, and cryptographic protection.

[operational twins?](#)

Episodes can usually be handled through relationships among constituent Facts plus lightweight metadata (identifier, time interval, episode type). A full operational twin analogous to `FactOperational` may be warranted if episode-level constructs become direct targets of cohort discovery.

Narratives are the weakest candidate for operational twins. They are interpretive objects whose value resides almost entirely in their content.

7 Consent Composition

A derived object—an Episode, a Narrative, a timeline view, an export—draws on multiple constituent Facts. Each constituent may be governed by a different consent policy. The system must compute a resulting policy for the derived object.

Default: intersection. The derived object can be disclosed only under conditions that satisfy every constituent’s policy. If a Narrative references Facts F_1 , F_2 , F_3 governed by policies P_1 , P_2 , P_3 , then the Narrative is governed by $P_1 \cap P_2 \cap P_3$.

Example: P_1 allows research use by academic institutions. P_2 allows research use by any party. P_3 allows clinical use only. Under intersection, the Narrative cannot be disclosed for research, because P_3 forbids it.

What about partial Episode / Narrative. Could contents be restricted to only F2 or F1+F2? Patient could allow partial reconstruction?

Union is never the right default. It would mean the least restrictive constituent governs the whole, violating the patient's intent on restricted Facts.

Implications.

1. **One restrictive Fact can lock a derived object.** A single highly-restricted Fact makes the entire Episode or Narrative inaccessible for otherwise-permitted purposes. The system should therefore support constructing derived objects that exclude specific Facts when the requesting purpose does not satisfy those Facts' policies. A borrower might receive a Narrative built from F_1 and F_2 but not F_3 .
2. **Policy granularity affects composition cost.** If every Fact has a bespoke policy, intersection is expensive to compute and hard to reason about. The recommendation is a small vocabulary of policy templates (e.g., "research-ok," "clinical-only," "fully-restricted") with per-Fact overrides in exceptional cases.
3. **Composition must precede materialization.** The system must determine the resulting policy before decrypting any constituent. The composition algebra operates over policy references in `FactOperational`, not over decrypted content. Otherwise the system decrypts Facts the requester is not allowed to see.
4. **Materialized vs. on-the-fly composition.** If a derived object is stable and shared (e.g., a published Narrative), it should carry a materialized composed policy. If it is reconstructed on demand, composition can be computed at access time.

Need result type?

```
pub enum DisclosureResult {
    Full(ResolvedEpisode),
    Partial {
        episode: ResolvedEpisode,    // built from authorized facts only
        excluded_count: usize,        // how many facts were excluded
        excluded_types: Vec<FactType>, // what categories, without identifying which
    },
    Denied,                          // no facts authorized for this purpose
}
```

8 Consent Revocation

A patient may revoke consent. Unless they are closing their account entirely and requesting full data deletion, revocation must be granular: scoped to specific Fact types, specific Episodes, specific borrowers, or specific purposes.

Scope of revocation. Revocation prevents future access. It does not retroactively un-disclose data already shared under a legitimate prior authorization. If a borrower received decrypted Facts last week and the patient revokes today, those Facts were legitimately accessed. The contract between the patient and Phoros governs what the borrower must do with data already in their possession (destroy, cease use, etc.), but the cryptographic architecture cannot un-ring that bell.

Mechanism. Revocation invalidates the key material or access grants associated with the revoked consent scope. If keys are scoped per-policy and the patient revokes a policy, the system stops issuing keys under that policy. Cached plaintexts in any operational system should have short TTLs and be invalidated on revocation events.

Contractual framing. The initial contract between the patient and Phoros specifies timelines and procedures for satisfying consent revocation requests. This includes the window within which revocation takes effect, the obligations of downstream borrowers, and the process for confirming deletion or cessation of use.

9 Provenance and Authorship

When a Fact records “authored by Dr. X,” what binds that claim? In the current design, the system verifies identity at the UI layer (e.g., authenticated session) and records authorship as a field within the Fact. Once written, authorship is encrypted with all other fields of the Fact and shares their protection.

This provides authentication but not non-repudiation. Dr. X cannot later prove they did not author the Fact, and Phoros cannot prove to a third party that Dr. X did, because the binding between identity verification and the encrypted record is mediated entirely by Phoros’s systems.

If third-party verifiability becomes a requirement—for legal proceedings, regulatory audits, or cross-institutional trust—the architecture would need digital signatures on Facts at authoring time. This is deferred.

10 Summary

The security architecture is organized around four commitments:

1. **Semantic alignment.** Protection boundaries follow FEN semantic units, not storage artifacts. Facts are the cryptographic unit. Episodes and Narratives inherit constraints from their constituents.
2. **Eager materialization.** FEN objects are created at ingestion. The object being encrypted and the object being governed are the same object.
3. **Transient plaintext.** Readable objects exist only in memory during authorized access. The system stores encrypted fragments and reconstructs semantic objects on demand through a policy-governed procedure.

4. **Operational separation.** `FactOperational` supports platform operations (indexing, discovery, policy evaluation) without decrypting payloads. Access to operational metadata is itself governed and audited.

Consent composition follows intersection by default. Revocation is granular and forward-looking. Key management and the signing mechanism for non-repudiation are deferred to subsequent documents.